HW1: Waypoint Navigation on the Rover

CSE 276A, Introduction to Robotics, UC San Diego · Soham Lokhande · October 2026

1. The problem

The robot gets a list of poses from `waypoints.txt`, one per line as `x, y, θ` (meters, radians):

```
0,0,0   1,0,0   1,1,1.57   2,1,0   2,2,−1.57   1,1,−0.78   0,0,0
```

It has to visit each one in order and face the given heading when it gets there. The only thing we can actually send the rover is a pair of unitless wheel commands `[L, R]` on the `motor_commands` topic, at 20 Hz. So the whole job is translating *"be at (1,1) facing 90°"* into *"send [-0.3, 0.3] for 1.23 s"*. Something has to do that translation, and that is the algorithm. To do it, it needs to know how the robot moves when you give it a command, and that is the kinematic model.

The rover has no wheel encoders and no position sensor. We also decided not to use the IMU (more on that in Section 4). So the robot can't see where it is; it can only estimate its position from what it was told to do. This is called dead reckoning, and it shaped every decision below.

2. Kinematic model

The rover is a skid-steer (differential-drive) vehicle, so we model it as a **unicycle**: a point with a heading θ that can move forward at speed v and rotate at rate ω:

```
ẋ = v·cos θ        ẏ = v·sin θ        θ̇ = ω
```

Intuition: if you face along +x, all your speed goes into x; face 90° and it all goes into y; in between it splits by cos/sin. The wheels connect to (v, ω) through `v_L = v − ω·W/2`, `v_R = v + ω·W/2`, where W is the wheel base.

We only ever use the two simplest cases of this model:

- **Drive straight** (ω = 0, L = R): the robot moves a distance d along its heading: `x += d·cos θ`, `y += d·sin θ`.
- **Turn in place** (v = 0, L = −R): only the heading changes: `θ += Δθ`.

Breaking every motion into one of these two keeps the model easy to calibrate and easy to reason about. A curved path mixes v and ω and you would need both to be accurate at the same time; with separate motions each one is calibrated on its own. We also learned that the textbook wheel-base relation does not hold well on this robot: the measured "effective" wheel base came out at 0.3–0.6 m depending on the command, much wider than the real wheel spacing, because skid-steer wheels slide sideways when turning in place. That is another reason to calibrate turning directly (angle vs time) rather than derive it from wheel speeds.

3. Algorithm: turn → drive → turn

Starting from the first waypoint as the known start pose, for each next waypoint (x_g, y_g, θ_g):

1. **Face the waypoint:** turn by `atan2(`y_g`−y, `x_g`−x) − θ`.
2. **Drive to it:** drive straight by `hypot(`x_g`−x, `y_g`−y)`, updating the pose with the unicycle model.
3. **Take the required heading:** turn by θ_g `− θ`.

Every turn angle is wrapped to $[-\pi, \pi]$ so the robot always turns the short way. Turns under ~1° and drives under 1 cm are skipped. The whole route is planned up front from the model; for our file it comes out to 14 motions:

Waypoint	Motions	Comment
(1, 0, 0)	drive 100 cm	already facing it
(1, 1, 1.57)	left 90°, drive 100 cm	arrives facing the required 90°
(2, 1, 0)	right 90°, drive 100 cm	
(2, 2, −1.57)	left 90°, drive 100 cm, right 180°	arrives facing +y, must face −y
(1, 1, −0.78)	right 45°, drive 141 cm, left 90°	travels at −135°, must face −45°
(0, 0, 0)	right 90°, drive 141 cm, left 135°	travels at −135°, must face 0°

Notice that at (2,2) and (1,1) the robot turns twice in a row: once to take the required heading, then again to face the next waypoint. That looks odd when you watch it, but it is exactly what the file asks for. The heading at a waypoint is not always the direction of the next leg.

4. Control algorithm

We use **open-loop (feedforward) control with a calibrated inverse model**. Each planned motion is turned into an `[L, R]` command and a duration:

```
drive d meters:    [0.196, 0.204]  for  t = d / DRIVE_SPEED + DRIVE_START_LOSS
turn  Δθ left:     [−0.3,  0.3 ]   for  t = Δθ / TURN_RATE_LEFT  + TURN_DELAY_LEFT
turn  Δθ right:    [ 0.3, −0.3 ]   for  t = Δθ / TURN_RATE_RIGHT + TURN_DELAY_RIGHT
```

The node publishes that command at 20 Hz for exactly that long (it stops on time rather than rounding up to the next 50 ms tick), sends stop, then **waits 1.5 s** before the next motion.

Why these choices

- **Fixed commands, control by time.** Instead of modeling speed as a function of command, the robot only ever uses one drive command and one turn command. Each one is calibrated as "how far does it go in t seconds". That makes the behavior as repeatable as possible: same command, same physics, only the time changes.
- **A pause between motions.** All calibration runs started from rest (after a 3 s countdown). In our first route test with a 0.5 s pause, the turn after a drive came up short because the robot was still settling. A pause recreates the conditions the model was calibrated under.

- **Slow turn command (0.3, not 0.5).** At 0.5 the robot turns ~280°/s, so a 90° turn lasts 0.33 s and a single 50 ms tick of the motor loop is ~14°. At 0.3 it turns ~80°/s and one tick is ~4°.
- **No IMU.** In early tests the IMU's yaw disagreed with the floor measurements (wrong sign and size), so we did not trust it for feedback. This is the main limitation, see Section 7.

5. Code

Everything is in the `robot_control` ROS 2 package. `motor_control` (from the template) is unchanged: it subscribes to `motor_commands` and forwards `{"T":1, "L":…, "R":…}` to the motor board over serial, with a 0.15 s safety stop if commands stop arriving. (`T` is just the message type in the board's protocol, 1 = set wheel speeds; it does not need calibrating.) Our waypoint node takes the keyboard node's place on the same topic.

File	What it does
`calibration.py`	All measured constants, with the raw data and fits in comments. `drive_command(d)` and `turn_command(Δθ)` convert a motion into `([L, R], seconds)`; `expected_drive/expected_turn` do the reverse.
`waypoint_follower.py`	ROS 2 node. Reads the waypoint file, `plan_motions()` builds the turn/drive/turn list with the unicycle model, then `execute()` plays each motion at 20 Hz and pauses. Parameters: `waypoints_file`, `settle_time`, `dry_run` (prints the plan without moving).
`calibrate.py`	Calibration tool: `calibrate drive T`, `calibrate left T`, `calibrate right T` use the exact commands from `calibration.py`, so calibration and navigation can never use different commands. Prints the model's prediction for comparison. `calibrate L R T` runs any raw command.

```
ros2 run robot_control motor_control                                  # terminal 1
ros2 run robot_control waypoint_follower --ros-args -p dry_run:=true  # terminal 2: plan only
ros2 run robot_control waypoint_follower                              # run it
```

6. Calibration

This took the most time and is where most of the learning happened. Method: place the robot on a taped start pose, run one command for a fixed time, measure distance and sideways drift with a tape measure, and measure turns with tape lines on the floor. Same floor as the route, full battery.

Round 1: model speed as a function of command (abandoned)

We first fit forward speed as a line in the command (0.2, 0.3, 0.4 for 3 s gave a clean linear fit) and measured turn rate at 0.5. This worked in isolation, but we then put tape on the wheels, which changed everything: 0.2 for 3 s went from 32 cm to 70 cm, turns became ~3× faster. Refitting the "speed vs command" model kept giving errors of 10+ cm, because a fit across commands mixes the speed with the fixed start/stop effects, and each command coasts differently. Turns at 0.5 also became unrepeatable (e.g. 0.38 s → 100°, 0.53 s → 175°).

Round 2: fixed commands, calibrated against time (final)

The key idea: **stop modeling the command, and only model time.** Fix one drive command and one turn command, then measure how far each goes at several durations and fit a line `distance = rate·(t − delay)`. Several durations are still needed even at fixed speed: the intercept captures the constant start-up/coast effect that makes distance not exactly proportional to time.

Drive [0.2, 0.2]			
time	**distance**	**drift**	**model**
1.5 s	41 cm	0	39 cm
2.5 s	64 cm	3 cm R	66 cm
3.5 s	93 cm	6 cm R	92 cm
4.5 s	118 cm	8 cm R	118 cm

Turn in place at 0.3 (degrees)				
time	**left**	**model**	**right**	**model**
0.6 s	40	41	35	53
1.2 s	85	88	110	113
1.8 s	130	134	175	172
2.4 s	180	181	215	232

- **Drive:** the line fit had essentially no intercept, so the speed is a fit through zero: **26.3 cm/s**, within 2 cm at every point.
- **Drift:** the robot curved right, about 8 cm per meter, so the left side was slightly faster. We added a trim that slows the left and speeds up the right by the same amount, `[0.196, 0.204]`, which keeps the average (and so the speed) the same. After the trim: 100 cm with 0 drift.
- **Left turn:** 77.5°/s with a ~0.07 s start-up delay, within ~2–4° of every run.
- **Right turn:** much noisier. Runs varied from 67 to 125°/s between points, so no single line fits well. We took the rate from the fit over all runs (99.1°/s) and set the delay so the line passes through a measured 90° check, since 90° is the most common right turn on the route.

Validation (commands computed from the final model, not fit to): drive 3.80 s → 100 cm, 0 drift; left 1.23 s → 90°; right 0.97 s → 90°. One important observation: running the *identical* left command twice gave 90° and 95°. So the turns have about **±3–5° run-to-run spread**, which is a property of the robot and floor, not of the calibration. No amount of refitting removes it.

7. Performance

We ran the full route with the calibrated model. What happened:

- (0,0) → (1,0): straight and on target.
- Left 90° and drive to (1,1): on target.

- Right turn toward (2,1): **turned ~95° instead of 90°**. The drive length was right, but the robot arrived slightly to the side of (2,1).
- Left 90° and drive toward (2,2): ended short of and beside (2,2), because it was still carrying the 5° heading error.
- The double turns at (2,2) and (1,1) happened as planned; the robot got close to (1,1) and then close to (0,0).

Distances were accurate; heading errors caused nearly all of the position error. The reason is simple geometry: a heading error of δ produces a sideways error of about `d·sin δ`. With δ = 5° that is ~9 cm per meter, and since the robot never re-measures its heading, the error carries into every later leg. Using the model on the run above: a 95° turn followed by a 1 m drive puts the robot near (2, 0.91) instead of (2, 1), and the next 90° turn then points it at 85° instead of 90°, landing near (2.09, 1.91) instead of (2, 2). That matches what we saw: "short of and beside (2,2)". The route has 8 turns, so even a few degrees of random error per turn adds up.

In short: the open-loop approach can be made accurate *on average* by calibration, but it cannot fix *random* per-run error, because nothing measures what actually happened.

What would make it better

1. **Close the loop on heading.** Turn until a sensor says the robot has turned 90°, instead of for a fixed time. The IMU's raw gyro rate (`angular_velocity.z`) integrated over a 1–2 s turn should be accurate to a degree or two even though the fused yaw looked wrong. The same heading signal could also steer during straight drives, replacing the hand-tuned trim.
2. **More repeats per calibration point.** Most points were measured once; 3–5 repeats would average out measurement noise and show the true spread.
3. **External position correction**, e.g. the camera and AprilTags, to reset the accumulated error. That is where the next assignments go.

8. Takeaways

- The kinematic model is only as good as its calibration, and the calibration only holds under the conditions it was measured in (floor, battery, tape on the wheels, starting from rest).
- Making the robot *repeatable* (fixed commands, pauses, a slower turn) mattered more than making the model fancy.
- Dead reckoning errors grow along the route, and heading errors are the most damaging. That is the motivation for feedback control and localization.